

AIMS

Calculus 1

Let $f(\mathbf{w})$ be a function of vector $\mathbf{w} \in \mathbb{R}^N$, i.e.,

$$f(\mathbf{w}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$$

$$f(\mathbf{w}) = \sigma(\mathbf{w}^T \mathbf{x})$$

$$\text{Where } \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$

Differentiate $f(\mathbf{w})$ concerning $\mathbf{w}$;

$$\nabla_{\mathbf{w}} f(\mathbf{w}) = f(\mathbf{w})(1 - f(\mathbf{w}))\mathbf{x}$$

therefore the gradient is

$$\nabla_{\mathbf{w}} f(\mathbf{w}) = f(\mathbf{w})(1 - f(\mathbf{w}))\mathbf{x}$$

Hessian or 2nd derivative

$$H = \nabla_{\mathbf{w}}^2 f(\mathbf{w})$$

$$\nabla_{\mathbf{w}} f(\mathbf{w}) = f(\mathbf{w})(1 - f(\mathbf{w}))\mathbf{x}$$

Using product rule

$$\nabla_{\mathbf{w}} f(\mathbf{w}) = (f(\mathbf{w})(1 - f(\mathbf{w})))^T \mathbf{x} \mathbf{x}^T$$

Since

$$\frac{d}{dw} f(w) (1 - f(w)) = f(w) (1 - f(w)) (1 - 2f(w))$$

Hessian becomes

The hessian matrix is given by

$$H = \nabla_w^2 f(w)$$

Differentiating the gradient as

$$H = f(w) (1 - f(w)) (1 - 2f(w)) XX^T$$

Since this is a scalar function

$$(1 - 2f(w))$$

Calculus 2.

Let $A \in \mathbb{R}^{N \times N}$ be a symmetric positive definite matrix and $b \in \mathbb{R}^N$ a vector. If

$X \in \mathbb{R}^N$, evaluate the integral $Z(A, b) = \int e^{-1/2 x^T A x - b^T x} dx$ as a function of A and b .

Solution;

Completing the square exponent can be written in a quadratic form by completing the square, we rewrite

$$-1/2 x^T A x + b^T x$$

As

$$-1/2 (x - A^{-1}b)^T A (x - A^{-1}b) + 1/2 b^T A^{-1}b$$

Thus, the integral transforms to

$$Z(A, b) = \int e^{-1/2 (x - A^{-1}b)^T A (x - A^{-1}b)} e^{1/2 b^T A^{-1}b} dx$$

The integral

$$\int e^{-1/2(x-\mu)^T A (x-\mu)} dx$$

$$\int e^{-1/2 x^T A x} dx = \sqrt{((2\pi)^N)/\det A}$$

Applying

$$Z(A, b) = e^{1/2 b^T A^{-1} b} \int e^{-1/2 (x - A^{-1} b)^T A (x - A^{-1} b)} dx$$

$$\text{Integral evaluates to } \sqrt{((2\pi)^N)/\det A}$$

, we get

$$Z(A, b) = \sqrt{((2\pi)^N)/\det A} e^{1/2 b^T A^{-1} b}$$

Final answer

$$Z(A, b) = \sqrt{((2\pi)^N)/\det A} e^{1/2 b^T A^{-1} b}$$

Probability 1

John throws a fair die with faces labeled 1 to 6.

- He gains 10 points if the die shows 1.
- He gains 1 point if the die shows 2 or 4.
- No points are allocated otherwise.

Let

X be the random variable describing John's gain at each throw. Determine the variance of X

Expected value $E(X)$;

$$E(X) = \sum x_i P(X = x_i)$$

$$E(X) = (10 \times 1/6) + (1 \times 2/6) + (0 \times 3/6)$$

$$E(X) = 10/6 + 2/6 + 0$$

$$= 12/6$$

$$E(X) = 2$$

Expected value of X^2 $E(X^2)$

$$E(X^2) = \sum x_i^2 P(x = x_i)$$

$$E(X^2) = (10^2 \times 1/6) + (1^2 \times 2/6) + (0^2 \times 3/6)$$

$$E(X^2) = (100 \times 1/6) + (1 \times 2/6) + 0$$

$$E(X^2) = 100/6 + 2/6 = 102/6 = 17$$

Variance Calculations $\text{Var}(X)$

$$\text{Var}(X) = E(X^2) - (E(X))^2$$

$$\text{Var}(X) = 17 - (2)^2$$

$$\text{Var}(X) = 17 - 4 = 13$$

$$\text{Var}(X) = 13$$

Probability 2

Each night, a princess is equally likely to sleep on anything from 6 to 12 mattresses. On half of the nights of the year, a pea is placed underneath the lowest mattress.

- If the pea is placed underneath 6 mattresses, she sleeps well 0 nights out of 10. She never sleeps!
- If the pea is placed underneath 7 mattresses, she sleeps well 1 night out of 10.
- If the pea is placed underneath 8 mattresses, she sleeps well 2 nights out of 10.
- If the pea is placed underneath 9 mattresses, she sleeps well 3 nights out of 10.
- If the pea is placed underneath 10 mattresses, she sleeps well 4 nights out of 10.
- If the pea is placed underneath 11 mattresses, she sleeps well 5 nights out of 10.
- If the pea is placed underneath 12 mattresses, she sleeps well 6 nights out of 10.

One morning, when her good friend Mr Bayes woke her up, she said that she had slept incredibly well that night!

What is the expected number of mattresses that she slept on that night?

Solution.

Events;

M_i Princess sleeps on i mattresses.

w Princess sleeps well.

$P_{mi} = 1/7$ for $i = 6, 7, 8, 9, 10, 11, 12$

Conditional probabilities

$$P(W|M_i)$$

$$P(W|M_6) = 0/10 = 0$$

$$P(W|M_7) = 1/10$$

$$P(W|M_8) = 2/10$$

$$P(W|M_9) = 3/10$$

$$P(W|M_{10}) = 4/10$$

$$P(W|M_{11}) = 5/10$$

$$P(W|M_{12}) = 6/10$$

Baye's Theorem

Expected number of mattresses given she slept well

$$E(M|w)$$

Probability of sleeping on i mattresses given she slept well

$$P(M_i|W)$$

$$P(M_i|W) = (P(W|M_i) \times P(M_i)) / P(W)$$

Where $P(W)$ is the probability of sleeping well.

Law of total probability

$$P(W) = \sum (P(W|M_i) \times P(M_i))$$

$$P(W) = (0 \times 1/7) + (1/10 \times 1/7) + (2/10 \times 1/7) + (3/10 \times 1/7) + (4/10 \times 1/7)$$

$$(5/10 \times 1/7) + (6/10 \times 1/7)$$

$$= (0 + 1 + 2 + 3 + 4 + 5 + 6) / 70$$

$$P(W) = 21/70 = 3/10$$

$P(M_i|W)$ for each i

$$P(M_6|W) = (0 \times 1/7) / 3/10 = 0$$

$$P(M_6|W) = (1/10 \times 1/7) / 3/10 = 1/21$$

$$P(M_6|W) = (2/10 \times 1/7) / 3/10 = 2/21$$

$$P(M_6|W) = (3/10 \times 1/7) / 3/10 = 3/21$$

$$P(M_6|W) = (4/10 \times 1/7) / 3/10 = 4/21$$

$$P(M_6|W) = (5/10 \times 1/7) / 3/10 = 5/21$$

$$P(M_6|W) = (6/10 \times 1/7) / 3/10 = 6/21$$

Expected value $E(M|W)$

$$E(M|W) = \sum i \times P(M_i|W)$$

$$E(M|W) = \{(6 \times 0) + (7 \times 1/21) + (8 \times 2/21) + (9 \times 3/21) + (10 \times 4/21) + (11 \times 5/21)$$

$$+ (12 \times 6/21)\}$$

$$= 217/21$$

$$= 10.33$$

Algebra 1

Suppose that M is an $m \times m$ matrix with entries $M_{ij} \geq 0$ such that $\sum_{i=1}^m M_{ij} = 1$ for each of the columns $j = 1, \dots, m$ show that 1 is an eigenvalue of M^k for any $k \in \mathbb{N}$.

Let M be a $m \times m$ matrix with entries $M_{ij} \geq 0$ such that $\sum_{i=1}^m M_{ij} = 1$ for each column $j = 1, \dots, m$

Let $v = \begin{bmatrix} 1 \\ 1 \\ \vdots \end{bmatrix}$ be a column vector of size $m \times 1$

$Mv = v$, which means v is an eigen vector of M corresponding to the eigenvalue 1

$$(Mv)_i = \sum_{j=1}^m M_{ij} v_j = \sum_{j=1}^m M_{ij} \cdot 1 = \sum_{j=1}^m M_{ij}$$

Consider vector $1 = \begin{bmatrix} 1 \\ 1 \\ \vdots \end{bmatrix} \in \mathbb{R}^m$

$M1 = \lambda 1$ for some scalar λ

let vector $w = \begin{bmatrix} 1 \\ 1 \\ \vdots \end{bmatrix}$

$$(M_w)_i = \sum_{j=1}^m M_{ij} w_j = \sum_{j=1}^m M_{ij} = 1$$

for all $i = 1, \dots, m$

Thus,

$$M_w = \begin{bmatrix} 1 \\ 1 \\ \vdots \end{bmatrix} = w$$

$\leftrightarrow$ this shows that w is an eigenvalue of M

Now we need to show that 1 is an eigenvalue of M_k for any $k \in \mathbb{N}$

if $M_v = \lambda v$, then $M^k v = \lambda^k v$

$M_w = 1w$, we have

$$M_k w = 1^k w = w$$

thus 1 is an eigenvalue of M^k for any $k \in \mathbb{N}$

Algorithms and Complexity 1

Consider n chords on a circle, each defined by its endpoints. Describe a $O(n \log n)$ -time algorithm to determine the number of pairs of chords that intersect inside the circle. (For example, if the n chords are all diameters that meet at the center, then the correct answer is $\binom{n}{2}$.) Assume that no two chords share an endpoint.

Solution

The key insight is to realize that two chords intersect if and only if their endpoints appear in an alternating order around the circle.

For example, if we have two chords A and B with endpoints A1, A2 and B1, B2 respectively, they intersect if the order of endpoints is

$A1 < B1 < A2 < B2$ or $B1 < A1 < B2 < A2$ when traversing the circle in a consistent direction (e.g., clockwise).

Absolutely! Let's break down this problem and craft an $O(n \log n)$ algorithm to solve it.

Problem Analysis

The core challenge is to efficiently count the intersecting pairs of chords. A naive approach of checking every pair of chords would take $O(n^2)$ time, which is too slow. We need a more clever strategy.

Key Insight

The key insight is to realize that two chords intersect if and only if their endpoints appear in an alternating order around the circle.

For example, if we have two chords A and B with endpoints A1, A2, and B1, B2 respectively, they intersect if the order of endpoints is $A1 < B1 < A2 < B2$ or $B1 < A1 < B2 < A2$ when traversing the circle in a consistent direction (e.g., clockwise).

Algorithm

Endpoint Representation: Represent each chord by its two endpoints.

Assign a unique index to each endpoint (e.g., from 1 to $2n$).

Sort Endpoints: Sort all $2n$ endpoints in clockwise order around the circle. This can be done in $O(n \log n)$ time using a standard sorting algorithm like merge sort or heapsort.

Intersection Counting:

Iterate through the sorted list of endpoints.

Maintain a counter for the number of "open" chords (chords whose starting endpoint has been encountered but not the ending endpoint).

For each endpoint:

If it's a starting endpoint:

Increment the open chord counter.

If it's an ending endpoint:

Decrement the open chord counter.

Add the current value of the open chord counter to the total intersection count. This is because when we encounter an ending endpoint, it means that the corresponding chord intersects with all the currently open chords.

Example:

Let's say we have 4 chords with endpoints:

A: (1, 5)

B: (2, 6)

C: (3, 7)

D: (4, 8)

Sorted Endpoints: (1, 2, 3, 4, 5, 6, 7, 8)

Iteration and Counting:

1 (A start): open_chords = 1

2 (B start): open_chords = 2

3 (C start): open_chords = 3

4 (D start): open_chords = 4

5 (A end): open_chords = 3, intersections += 3

6 (B end): open_chords = 2, intersections += 2

7 (C end): open_chords = 1, intersections += 1

8 (D end): open_chords = 0, intersections += 0

Total intersections = 3 + 2 + 1 + 0 = 6

Time Complexity Analysis

- Sorting the endpoints: $O(n \log n)$
- Iterating and counting: $O(n)$

Therefore, the overall time complexity of the algorithm is $O(n \log n)$.

